WikangAI: Philippine Language Identification System

Kirk Henrich Cambel Gamo, Clarence Anthony Bolivar, Jullian Bilan, Jan Floyd Vallota

College of Information and Communications Technology
West Visayas State University
Luna St., La Paz, Iloilo City 5000

***Abstract*—WikangAI is a web-based language identification system focused on Philippine languages. It classifies text into Tagalog, Cebuano, Hiligaynon, or Ilocano using a character-level CNN and a preprocessing pipeline designed for low-resource language variation. The system targets real-world inputs such as news headlines and short user messages, where spelling variation and shared vocabulary are common. This document summarizes the technical aspects, functional requirements, and roadmap of the developed system.**

I. Overview and Introduction

A. App Name

WikangAI

B. Purpose

The Philippines is a multilingual country with many low-resource languages. WikangAI addresses the lack of reliable language identification tools for regional Philippine languages, especially Hiligaynon, by providing a web-based classifier that supports Tagalog, Cebuano, Hiligaynon, and Ilocano. The system is designed as a lightweight, local deployment that can be embedded in downstream applications such as content routing, localization pipelines, and language-specific NLP workflows.

C. Technology Stack (with Versions)

The system is implemented using a modern Python ML stack and a lightweight web frontend. All components are locally deployable to meet classroom and offline constraints.

Backend and ML:

- Python 3.13
- FastAPI 0.136
- TensorFlow 2.21
- Keras 3.14
- scikit-learn 1.3+
- pandas 2.0+
- numpy 2.0+
- joblib 1.3+

Data and Utilities:

- beautifulsoup4 4.12+
- datasets 2.0+
- requests 2.31+

Frontend:

- HTML5, CSS3, Vanilla JavaScript (ES6)

Model Artifacts:

- CNN model saved in HDF5 format (`.h5`)
- Tokenizer and label encoder serialized with joblib (`.pkl`)

II. Functional Requirements

This section outlines the functional scope implemented in the prototype, including user-facing features, preprocessing logic, and API behaviors.

A. Core Features with Descriptions

- **Multilingual Text Identification:** Accepts user input and classifies the text into one of four supported languages with a softmax score per class.
- **Automated Preprocessing Pipeline:** Normalizes unicode, removes noise (URLs, mentions, hashtags), strips digits and excessive punctuation, and standardizes Hiligaynon spelling variations.
- **Confidence Scoring Display:** Returns a probability distribution across all languages and renders it as a bar chart in the UI.
- **API Communication:** Exposes a REST endpoint (`/predict`) for inference, allowing direct use from scripts or the browser UI.
- **Web Interface:** Provides a single-page interface with a textarea for input, a prediction card, and visual confidence bars.

B. Libraries/Packages

The following libraries are required to run training, evaluation, and inference in the system:

- **FastAPI:** REST API framework for model inference.
- **TensorFlow/Keras:** Training and inference for the character-level CNN.
- **scikit-learn:** Baseline model and dataset utilities.
- **pandas/numpy:** Data manipulation and preprocessing.
- **joblib:** Serialization of model artifacts.

C. Technical Workflow

Input Flow: User text is preprocessed, tokenized at the character level, padded to a fixed length, and passed to the CNN for inference. The predicted label and confidence scores are returned as JSON to the frontend.

Modeling Details: The CNN uses an embedding layer, two Conv1D layers with ReLU activations, max pooling, and a dense classifier head. Training uses early stopping on validation accuracy to avoid overfitting.

III. Future Enhancements and Roadmap

This roadmap focuses on improving robustness for short inputs, extending language coverage, and increasing accuracy using stronger architectures.

A. Known Limitations

- **Orthographic Similarity:** Cebuano and Hiligaynon share many words, causing confusion in short inputs.
- **Short Text Sensitivity:** Very short phrases reduce classification reliability.
- **Domain Shift:** Performance can drop when input is very informal or heavily code-mixed.
- **Fixed Context Window:** Padding to a fixed length can truncate long inputs and reduce context.

B. Planned Features and Improvements

- **Token-level Language Identification:** Identify language per token for mixed-language text.
- **Expand Language Coverage:** Add Kinaray-a, Akeanon, and other regional languages.
- **Model Enhancements:** Multi-kernel CNNs or transformer-based models for higher accuracy.
- **Robust Evaluation:** Add a fixed holdout test set and per-class error analysis.
- **Better Calibration:** Improve confidence calibration using temperature scaling or Platt scaling.